

A logical framework for dependent sort specifications and their finite models

Ambroise LAFONT

September 26, 2025

We present a logical framework for specifying dependent sort specifications (which we call *schemas*) and their finite models (which we call *complexes*, as in [1]). This is inspired by Uemura’s logical framework for SOGATs [2].

Example 1 *A sort specification for categories can be given as follows:*

- $ob : \mathbf{Set}$
- $hom : ob \rightarrow ob \rightarrow \mathbf{Set}$
- $id : \prod x : ob. hom(x, x) \rightarrow \mathbf{Set}$
- $comp : \prod x, y, z : ob. hom(x, y) \rightarrow hom(y, z) \rightarrow hom(x, z) \rightarrow \mathbf{Set}$

Here is an example of a finite model, or complex:

- $a : ob$
- $b : ob$
- $f : hom(a, a)$
- $g : hom(b, a)$
- $id_f : id(f)$

Clearly those specifications look like some contexts of some domain specific type theory. We sketch how to make it formal here.

Disclaimer I am not sure how original this is; it may just be a reformulation of Makkai’s FOLDS [3].

1 Logical framework

1.1 Informal description

A schema is intuitively a context of (dependent) sort declarations where each variable has a type of the shape $\prod_{a_1:A_1} \dots \prod_{a_n:A_n} \mathbf{Set}$. We may write this type as $\Delta \rightarrow \mathbf{Set}$, where Δ is the context $(a_1 : A_1, \dots, a_n : A_n)$. Since all sort declarations end with " $\rightarrow \mathbf{Set}$ ", we may as well omit this part. To sum up, a schema is a context $x_1 : \Delta_1, \dots, x_p : \Delta_p$. Now, each Δ_i describes a complex (or finite model) over the schema $(x_1 : \Delta_1, \dots, x_{i-1} : \Delta_{i-1})$.

Example 2 *Let us reformulate the beginning of Example 1 with this notation. The models of $ob : ()$ are sets. We have $hom : (a : ob, b : ob)$. Note that $(a : ob, b : ob)$ can be interpreted as a finite model of $ob : ()$, that is, the finite set with two elements a and b . A model of ob, hom is a graph. Next, we have $id : (x : ob, f : hom(x, x))$. Again, $(x : ob, f : hom(x, x))$ can be interpreted as the finite graph consisting of one looping arrow.*

1.2 Judgements

- $\Gamma \vdash$ means that Γ is a schema.
- $\Gamma | \Delta \vdash$ means that Δ is a complex over the schema Γ ;
- $\Gamma | \Delta \vdash A$ means that A is a sort in context $\Gamma | \Delta \vdash$.
- $\Gamma | \Delta \vdash \sigma \Rightarrow \Delta'$ means that σ is a substitution from the complex Δ to the complex Δ' .
- $\Gamma \vdash t : \Delta$ means that the term t has type Δ in Γ . In fact, recalling our shortcut notation, we shall actually think of t as a term of type $\Delta \rightarrow \mathbf{Set}$.
- $\Gamma | \Delta \vdash t : A$ means that t is a term of type A .

Note that we have two distinct judgements for terms: the first one lives in a schema, while the second one lives in a complex depending of some schema.

1.3 Typing rules

There are the obvious context extension rules for schemas and complexes.

$$\frac{\Gamma | \Delta \vdash}{\Gamma, x : \Delta} \quad \frac{\Gamma | \Delta \vdash A}{\Gamma | \Delta, x : A \vdash}$$

We have the obvious variable rules for schemas and complexes

$$\frac{(x : \Delta) \in \Gamma}{\Gamma \vdash x : \Delta} \quad \frac{(x : A) \in \Delta}{\Delta \vdash x : A}$$

In fact this is the only rule for terms! In other words, **all terms are variables**.

We omit the rules for building substitutions between complexes (substitutions are suitable lists of terms) which are standard. The most important rule is the sort formation rule:

$$\frac{\Gamma \vdash t : \Delta' \quad \Gamma | \Delta \vdash \sigma \Rightarrow \Delta'}{\Gamma | \Delta \vdash t(\sigma)}$$

This rule makes sense if we think of $t : \Delta'$ as $t : \Delta' \rightarrow \mathbf{Set}$.

Remark 1 *The notational convention consisting in omitting the part " $\rightarrow \mathbf{Set}$ " may look confusing. The main benefit is that we don't have to give $\mathbf{Set}$ a meaning, thus it makes the semantics of the logical framework simpler.*

1.4 Rocq formalisation

Matthieu Piquerez has a formalization of the logical framework based on raw syntax based on lists of natural numbers, with

- well-formed boolean predicates;
- a Ltac DSL as an interface to build contexts;
- pushouts of complexes (morphisms are substitutions between complexes).

1.5 Semantics

It is well-known that schemas correspond to certain kind of finite categories called *simple*. Then, the category of models of a schema is equivalent (but not isomorphic) to a category of functors from the corresponding simple category to $\mathbf{Set}$.

Example 3 *A model of $ob : (), hom : (a : ob, b : ob)$ is a graph, that is: a set of vertices ob and a set of edges hom indexed by $ob \times ob$. The corresponding simple category is $\Rightarrow$, and a functor to $\mathbf{Set}$ consists of a set of objects ob , a set of arrows hom , and two maps $dom, codom : hom \rightarrow ob$.*

Remark 2 *As argued before, a complex can be thought of as a finite model of a schema, or a "finite" functor F from a simple category C to $\mathbf{Set}$. The length of the complex is exactly the cardinal of $\sum_{c \in C} F(c)$. For example, the complex $a : ob, b : ob, f : hom(a, b)$ is of size 3.*

A schema Γ can be interpreted as:

- a simple category C_Γ ;
- a category of models $\llbracket \Gamma \rrbracket$, which is equivalent to $[C, \mathbf{Set}]$,

A complex $\Gamma|\Delta$ can be interpreted as an object $\llbracket \Delta \rrbracket$ of the category of models of Γ , or more abstractly, as a functor¹ from Γ to **Set**. Indeed, any object induces such a functor $y\Delta$ by the Yoneda embedding.

The context $\llbracket \Gamma, x : \Delta \rrbracket$ can be interpreted as the category $(\gamma : \llbracket \Delta \rrbracket) \times \mathbf{Set}^{\text{hom}(\llbracket \Delta \rrbracket, \gamma)}$.

Again, a type $\Gamma|\Delta \vdash A$ can be interpreted as an object $\llbracket A \rrbracket$ of the coslice category $\llbracket \Delta \rrbracket / \llbracket \Gamma \rrbracket$. Then, we can define $\llbracket \Gamma|\Delta, x : A^+ \rrbracket$ as the coslice category $\llbracket A \rrbracket / \llbracket \Gamma \rrbracket$.

The interpretation of terms is then derived: they are sections of the projections from extended contexts to the base context.

Substitution between schemas must be interpreted as adjunctions between categories of models, so that substituted complexes are interpreted by applying the left adjoint to the original complex. The most interesting part is the interpretation of the application rule (TODO: provide some details.).

This interpretation yields a model of the logical framework. This model which should actually be equivalent to the syntactic model. In particular, any simple category should induce a syntactic schema, and any finite model should induce a syntactic complex.

2 A single-context logical framework

In the provided semantics, there is some redundancy, in the sense that the interpretation of a complex is very similar to the interpretation of a type. In this section, we present a single-context logical framework, with simpler semantics, where contexts are no longer of the shape $\Gamma|\Delta$, but just some Γ . Contrary to the previous logical framework which cleanly separates schemas and complexes, the context can now interleave sort declarations and complex declarations, so that the following context is valid: $ob : \mathbf{Set}, a : ob, hom : ob \rightarrow ob \rightarrow \mathbf{Set}$. The interest of this single version is that:

- The semantics of this logical framework is simpler in some respect;
- Any model of this logical framework induces a model of the previous one².

Therefore, we can exhibit a model of the original 2-context version from a model of the 1-context version.

2.1 Polarities

A context is a list of variable declarations $x_1 : A_1^{+/-}, \dots, x_n : A_n^{+/-}$, that is, each variable x_i has an associated type A_i and a "polarity" $+$ or $-$. A negative declaration $x : A^-$ means that we should think of x as of type $A^+ \rightarrow \mathbf{Set}$, so that

¹This is a route followed by [4]

²In particular, by initiality/recursion, we have a syntactic translation from the two-context logical framework to the single-context logical framework.

we have the following application rule

$$\frac{\Gamma \vdash t : A^- \quad \Gamma \vdash \delta : A^+}{\Gamma \vdash t(\delta)}$$

We have two extension rules for contexts, depending on the chosen polarity

$$\frac{\Gamma \vdash A}{\Gamma, x : A^+ \vdash} \quad \frac{\Gamma \vdash A}{\Gamma, x : A^- \vdash}$$

Note that we have two term judgements $\Gamma \vdash t : A^+$ and $\Gamma \vdash t : A^-$, depending on the polarity.

2.2 Type formers

We assume a unit type and a sigma type with domain in a positive type.

$$\frac{}{\Gamma \vdash 1} \quad \frac{\Gamma \vdash A^+ \quad \Gamma, a : A^+ \vdash B}{\Gamma \vdash \sum_{x:A} B}$$

Such that we have projections and pairing as usual for $(\sum_{x:A})^+$, and a unique term $\Gamma \vdash * : 1^+$.

2.3 Example: categories

The context for categories is: $ob : 1^-, hom : (ob \times ob)^-, id : (\sum_{x:ob} hom(x, x))^-$

2.4 Semantics

The semantics is similar to the 2-context version, although we can no longer directly interpret a context as a simple category since we can interleave positive and negative declarations.

References

- [1] C. Leena Subramaniam, “From dependent type theory to higher algebraic structures,” Theses, Université de Paris, Sep. 2021, PhD Thesis, 155 pages, in English (with English and French introductions). [Online]. Available: <https://theses.hal.science/tel-03369103>
- [2] T. Uemura, “A general framework for the semantics of type theory,” *Math. Struct. Comput. Sci.*, vol. 33, no. 3, pp. 134–179, 2023. [Online]. Available: <https://doi.org/10.1017/s0960129523000208>
- [3] M. Makkai, “First order logic with dependent sorts, with applications to category theory.” [Online]. Available: <https://api.semanticscholar.org/CorpusID:2786865>
- [4] T. Altenkirch, P. Capriotti, G. Dijkstra, and F. N. Forsberg, “Quotient inductive-inductive types,” *CoRR*, vol. abs/1612.02346, 2016. [Online]. Available: <http://arxiv.org/abs/1612.02346>